

Assignment 2: ANN for Digit Recognition & Breast Cancer Classification

- **Objective 1:** Using the MNIST database, design Back-Propagation ANNs for the recognition of the ten digits. The resolution of the original images is reduced to 7x7.
- **Class03f1_MNIST.ipynb**
- **Objective 2:** Using the breast cancer database from Wisconsin University, develop ANNs for the binary classification based on a number of input features.
- **Class03f2_Breast-Cancer-Classification.ipynb**

- **Part A: MNIST** is a famous handwritten digits dataset often used for demonstrating deep learning. The MNIST dataset comprises 60,000 training examples and 10,000 test examples of the handwritten digits 0–9, formatted as 28x28-pixel monochrome images.



label = 5



label = 0



label = 4



label = 1



label = 9



label = 2



label = 1



label = 3



label = 1



label = 4



label = 3



label = 5



label = 3



label = 6



label = 1



label = 7



label = 2



label = 8

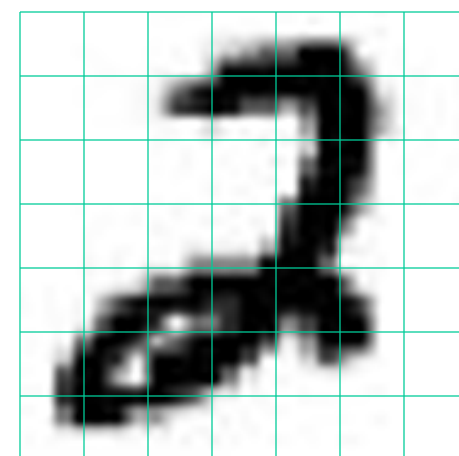


label = 6



label = 9

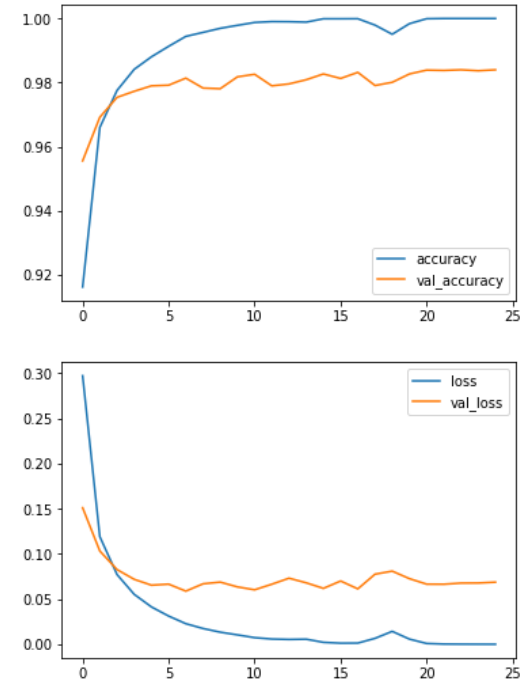




Further reduced to
7 x 7 pixel image

The tasks in Part A:

1. With reference to the file Class03f1_MNIST.ipynb, in Example 2 and 3, generate the graph of training accuracy and val_accuracy. In addition, get the graph of training loss and val_loss. The graph below is an example. Give comments on the graph.
2. Using Example 1 as a reference, resize the image to a resolution of 7x7 and use as inputs. Use the python module **cv2**, which has a function called **resize**. When training the ANN, your network would have 49 inputs and 10 outputs. **Show samples of the resized image** in your submission.
3. With input image dimension of only 7x7, develop and train BP ANN with **one** hidden layer. Vary the number of hidden neurons. Document your results in a table.
4. Similarly, develop and train BP ANN with **two** hidden layers. Vary the number of hidden neurons in the two hidden layers. Document your results in a table.
5. It would be interesting to analyze how the number of parameters in your neural network relate to the classification accuracy.



- **Part B:** This part of the assignment aims for classifying cancers (malignant or benign) based on ANN machine learning method. A breast cancer dataset from Wisconsin University is used. The dataset contains 569 samples and 30 features computed from digital images.

Useful python packages:

- scikit-learn is a library for machine learning algorithms
- Keras is a library for deep learning algorithms
- Pandas is used for data processing
- Seaborn is used for data visualization

If using jupyter notebook, in Command Prompt, you should install the followings if necessary:

- (change directory to your current folder)
- `pip install pandas`
- `pip install scikit-learn`
- `pip install seaborn`

```
import numpy as np
```

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
# Loading data from the .csv file
```

```
data_raw = pd.read_csv('BCW_dataset.csv', delimiter=',', header=0, index_col=None)
```

```
# Head method show first 5 rows of data
```

```
print(data_raw.head())
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

	... texture_worst	perimeter_worst	area_worst	smoothness_worst	\
0	... 17.33	184.60	2019.0	0.1622	
1	... 23.41	158.80	1956.0	0.1238	
2	... 25.53	152.50	1709.0	0.1444	
3	... 26.50	98.87	567.7	0.2098	
4	... 16.67	152.20	1575.0	0.1374	

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	\
0	0.6656	0.7119	0.2654	0.4601	
1	0.1866	0.2416	0.1860	0.2750	
2	0.4245	0.4504	0.2430	0.3613	
3	0.8663	0.6869	0.2575	0.6638	
4	0.2050	0.4000	0.1625	0.2364	

	fractal_dimension_worst	Unnamed: 32
0	0.11890	NaN
1	0.08902	NaN
2	0.08758	NaN
3	0.17300	NaN
4	0.07678	NaN

```
[5 rows x 33 columns]
```

Ten real-valued features are computed for each cell nucleus.

The mean, standard error and "worst" or largest (mean of the three largest values) of these features were computed for each image, resulting in 30 features.

For instance, field 3 is Mean Radius, field 13 is Radius SE, field 23 is Worst Radius.

The full 30 features:

['mean radius' 'mean texture' 'mean perimeter' 'mean area' 'mean smoothness' 'mean compactness' 'mean concavity' 'mean concave points' 'mean symmetry' 'mean fractal dimension' 'radius error' 'texture error' 'perimeter error' 'area error' 'smoothness error' 'compactness error' 'concavity error' 'concave points error' 'symmetry error' 'fractal dimension error' 'worst radius' 'worst texture' 'worst perimeter' 'worst area' 'worst smoothness' 'worst compactness' 'worst concavity' 'worst concave points' 'worst symmetry' 'worst fractal dimension']

Selected 9 features:

worst concave points ; radius error ; worst smoothness ;
worst symmetry ; worst texture ; concave points error ;
mean symmetry ; compactness error ; concavity error


```
import seaborn as sns
import matplotlib.pyplot as plt

# Split dataset into training (80%) and test (20%) set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

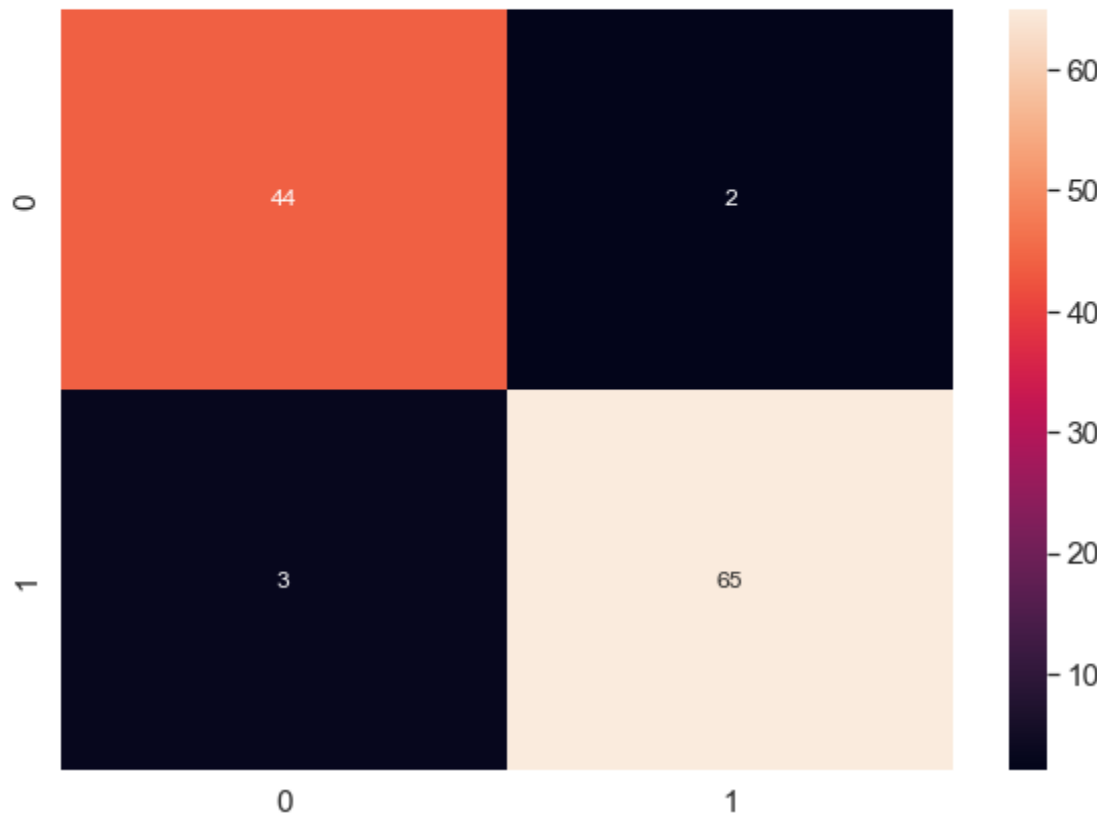
# Normalization of the data
X_train_n = (X_train-X_train.mean()/(X_train.max()-X_train.min()))
X_test_n = (X_test-X_train.mean()/(X_train.max()-X_train.min()))

print(X_train_n.shape)
print(y_train.shape)
print(y[0:3])

from sklearn.preprocessing import StandardScaler
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Dropout
from keras.layers import Flatten
from keras.layers import Activation
```

```
from sklearn.metrics import confusion_matrix
```

```
len(y_pred) 114  
[0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 1 0 0 0 0 0 1 1 1 0 1 1 0 1 0 1 0 1 0 1  
 0 1 0 0 1 0 1 1 0 1 1 1 0 0 0 0 1 1 1 1 1 1 0 0 0 1 1 0 1 0 0 0 1 1 0 1 0  
 0 1 1 1 1 1 0 0 0 1 0 1 1 1 0 0 1 1 1 0 1 1 0 1 1 1 1 1 1 1 0 1 0 1 1 1 1  
 0 0 1]  
len(y_test_values) 114  
[0 1 1 1 1 1 1 1 1 1 1 1 1 1 0 1 0 0 0 0 0 0 1 1 0 1 1 0 1 0 1 0 1 0 1 0 1  
 0 1 0 0 1 0 1 1 0 1 1 1 0 0 0 0 1 1 1 1 1 1 0 0 0 1 1 0 1 0 0 0 1 1 0 1 0  
 0 1 1 1 1 1 0 0 0 1 0 1 1 1 0 0 1 0 1 0 1 1 0 1 1 1 1 1 1 1 0 1 0 1 0 0 1  
 0 0 1]
```



Confusion Matrix is a performance measurement for machine learning classification problem, where output can be two or more classes.



TP (True Positive) – you predicted positive and it is true,
FP (False Positive) – you predicted positive and it is false,
FN (False Negative) – you predicted negative and it is false,
TN (True Negative) – you predicted negative and it is true.

```
from sklearn.metrics import recall_score  
from sklearn.metrics import f1_score  
from sklearn.metrics import precision_score
```

PRECISION, RECALL & F1 SCORE

$$\text{Precision} = \frac{TP}{TP + FP}$$

e.g. TP = 65; FP = 3 : Precision = (65/(65+3)) = 0.956

When we predict it is positive, how much we predicted corrected ?
Out of all the classes, how much we predicted correctly.

$$\text{Recall} = \frac{TP}{TP + FN}$$

e.g. TP = 65; FN = 2 : Recall = (65/(65+2)) = 0.97

Out of all the positive classes, how much we predicted correctly.

$$F = \frac{2}{\left(\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}\right)}$$

f1_score = 0.963

F1-score is the harmonic mean of the precision and recall.

Class03f2 Breast-Cancer-Classification.ipynb

```
In [1]: #from google.colab import drive
        #drive.mount('/content/drive')

        # Filename: Class04f2_Breast-Cancer-Classification.ipynb
        # if necessary:
        # pip install scikit-learn
        # pip install pandas
```

```
In [261]: # Colab import libraries
          # Click only in Colab

import sklearn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.preprocessing import StandardScaler
from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Dropout
```

```
In [34]: # Jupyter Notebook import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
from tensorflow.keras.layers import Flatten

from tensorflow.keras.layers import Conv2D
from tensorflow.keras.layers import MaxPooling2D

from tensorflow.keras.utils import to_categorical
from tensorflow.keras import backend as K
```

Colab

or

Jupyter
notebook

```
# Loading data from the .csv file if in Colab  
# data_raw = pd.read_csv('/content/drive/MyDrive/Colab Notebooks/BCW_dataset.csv', c  
  
data_raw = pd.read_csv('BCW_dataset.csv', delimiter=',', header=0, index_col=None)  
# Head method show first 5 rows of data  
print(data_raw.head())
```

Readin from
Jupyter
notebook

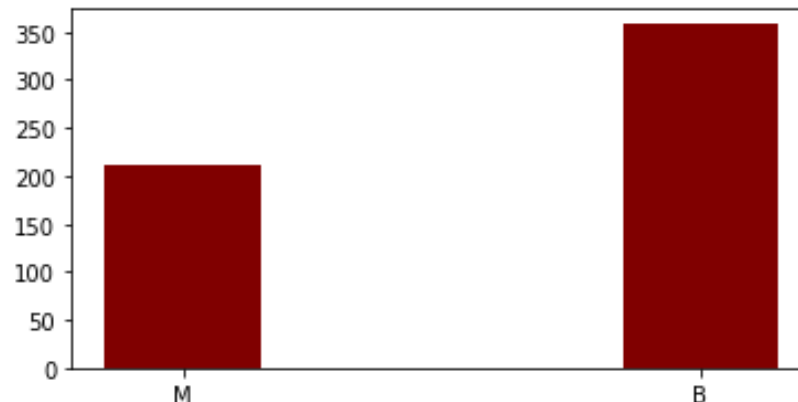
```
# Drop unused columns  
drop_columns = ['Unnamed: 32', 'id', 'diagnosis']  
X = data_raw.drop(drop_columns, axis=1)  
  
# Convert strings -> integers  
d = {'M': 0, 'B': 1}  
  
# Define features and Labels  
y = data_raw['diagnosis'].map(d)
```

```
# Plot number of M - malignant and B - benign cancer
```

```
B, M = y.value_counts()  
data = {'M':M, 'B':B}  
courses = list(data.keys())  
values = list(data.values())  
print(courses)  
print(values)
```

```
fig = plt.figure(figsize = (6, 3))  
plt.bar(courses,values,color='maroon',width=0.3)  
plt.show()  
print('Number of benign cancer: ', B)  
print('Number of malignant cancer: ', M)
```

```
['M', 'B']  
[212, 357]
```



```
Number of benign cancer: 357  
Number of malignant cancer: 212
```

```

# Example
# One hot encoding
encode_array = OneHotEncoder()
Y = encode_array.fit_transform(y[:, None]).toarray()
print('y[0:5] = ',y[0:5])
print(' ')
print('First five one hot encoded output: ')
print(Y[0:5])

# Scale data to have mean 0 and variance 1
# which helps the convergence of the neural network
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print(' ')
print('First five scaled input features: ')
print(X_scaled[0:5,0:4])

```

```
y[0:5] = 0    0
```

```
1    0
```

```
2    0
```

```
3    0
```

```
4    0
```

```
Name: diagnosis, dtype: int64
```

```
First five one hot encoded out
```

```
[[1. 0.]
```

```
[1. 0.]
```

```
[1. 0.]
```

```
[1. 0.]
```

```
[1. 0.]]
```

```
First five scaled input features:
```

```
[[ 1.09706398 -2.07333501  1.26993369  0.9843749 ]
```

```
[ 1.82982061 -0.35363241  1.68595471  1.90870825]
```

```
[ 1.57988811  0.45618695  1.56650313  1.55888363]
```

```
[-0.76890929  0.25373211 -0.59268717 -0.76446379]
```

```
[ 1.75029663 -1.15181643  1.77657315  1.82622928]]
```

```

# Split dataset into training (75%) and test (25%) set
X_train, X_test, Y_train, Y_test = train_test_split(X_scaled, Y, test_size=0.25, random_state=2)

print('X.shape = ', X.shape)
print('Y.shape = ', Y.shape)
num_features = X.shape[1]
num_classes = Y.shape[1]
print('num_features = ', num_features)
print('num_classes = ', num_classes)

# Normalization of the data
X_train_n = (X_train-X_train.mean())/(X_train.max()-X_train.min())
X_test_n = (X_test-X_train.mean())/(X_test.max()-X_test.min())
print('X_train_n.shape = ', X_train_n.shape)
print('X_test_n.shape = ', X_test_n.shape)
print('Y_train.shape = ', Y_train.shape)
print('Y_test.shape = ', Y_test.shape)
print('Y_train[0:3] is ')
print(Y_train[0:3])

```

```

X.shape = (569, 30)
Y.shape = (569, 2)
num_features = 30
num_classes = 2

```

```

# define the model structure
# Dense : a regular fully connected layer

# The following is a simple one-hidden layer ANN;
#
# Define the number of hidden nodes
num_hidden_nodes = 50

def model_1():
    # create model
    # .....|

    # .....
    # Choice of optimizer: adam (adaptive moment estimation),
    # or sgd (Stochastic gradient descent),
    return model

# build the model
model = model_1()

```

Keras Framework example

```

# Training of the ANN
# Use of validation data
#
#
history = model.fit( ....)|

# Testing of the trained ANN with X_test_n
score = model.evaluate(....)
print('Test loss:', score[0])
print('Test accuracy:', score[1])

```

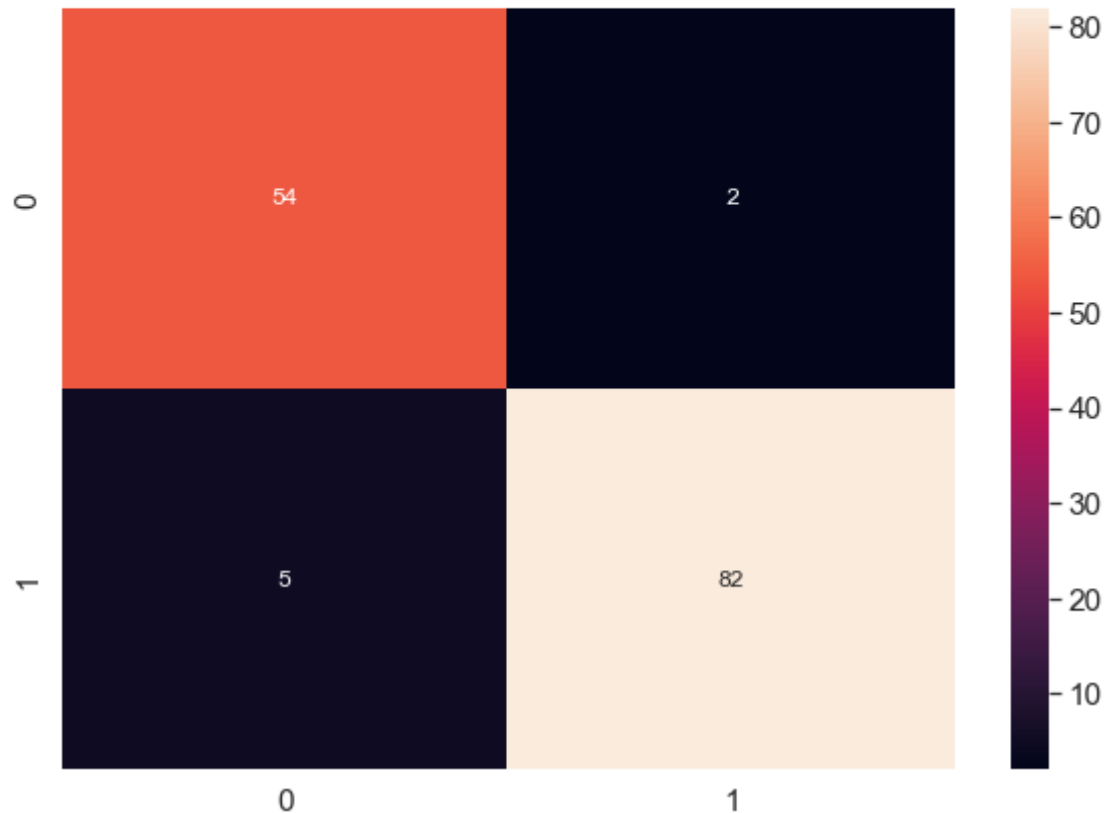
In Training: X_{train_n} → Y_{train}

In Testing: X_{test_n} → Y_{test} (desired) → y_{test}
(argmax)

→ Y_{ANN} (from ANN) → y_{ANN}
(argmax)

```
"  
# ANN to predict the entire test data  
# ANN output is Y_ANN, with input X_test_n. Desired output is Y_test  
Y_ANN=model.predict(X_test_n)  
  
# Use of np.argmax : 0 is first position; 1 is second position  
y_ANN=np.argmax(Y_ANN, axis=1)  
y_test=np.argmax(Y_test, axis=1)  
print(y_test.shape,y_ANN.shape)  
print('=====')  
print('Y_ANN ', Y_ANN[0:5,:])  
print('y_ANN ', y_ANN[0:5])  
print('Y_test ', Y_test[0:5])  
print('y_test ', y_test[0:5])  
  
print('-----')  
print('Confusion Matrix is ')  
print(confusion_matrix(y_test,y_ANN))  
print(precision_score(y_test,y_ANN))  
print(recall_score(y_test,y_ANN))  
print(f1_score(y_test,y_ANN))
```

```
: # More elegant display of confusion matrix
cm = confusion_matrix(y_test,y_ANN)
df_cm = pd.DataFrame(cm, range(2),range(2))
plt.figure(figsize=(10,7))
sns.set(font_scale=1.4)#for label size
cm_plot = sns.heatmap(df_cm, annot=True, fmt='n', annot_kws={"size": 12})
```



```
# To show the performance of the ANN during training
# The information has already been saved in "history"
```

The tasks for the Assignment:

Part A

Task description given earlier. Use at most **TWO pages for Part A.**

Part B

- a. Develop an ANN with **one hidden layer** for binary classification using all the **30 features** as input. (Use 2 output units, with loss='categorical_crossentropy') Use 75% of the dataset for training, and the rest for testing.(metrics=['accuracy'])
- b. Develop an ANN with **three hidden layers** for binary classification using all the **30 features** as input. (Use 2 output units, with loss='categorical_crossentropy') Use 75% of the dataset for training, and the rest for testing.
- c. Analyze your test results of items a and b using confusion_matrix, precision_score, recall_score and f1_score.
- d. Develop an ANN with **one hidden layer** for binary classification using **9 features** as input. (Use 2 output units, with loss='categorical_crossentropy') Use 75% of the dataset for training, and the rest for testing. (metrics=['accuracy'])
- e. Develop an ANN with **three hidden layers** for binary classification using **9 features** as input. (Use 2 output units, with loss='categorical_crossentropy') Use 75% of the dataset for training, and the rest for testing.
- f. Analyze your test results of items d and e using confusion_matrix, precision_score, recall_score and f1_score.

It would be interesting to analyze how the number of parameters in your neural network relate to the classification accuracy.

In both Part A & B, you should provide an analysis and a discussion of your results.
Watch out for overfitting when you use a large number of neurons in the hidden layer.

In Part B, use only **ONE** page to report your results of Task a, b and c.

In Part B, use only **ONE** page to report your results of Task d, e and f.

Your submission of one report:

Page 1: A cover page with your name and an **abstract of Part A and B.**

Page 2 & 3: Two pages from Part A. All figures, tables and text must be on two pages only.

Page 4/ **Part B: One page on your work of Task a, b and c.
All figures, tables and text must be on one page only.**

Page 5/ **Part B: One page on your work of Task d, e and f.
All figures, tables and text must be on one page only.**

Template file given: Class03g_Assignment2_Template.docx

Individual report. Only ZIP file allowed. Submit online.

Multiple files in the zip file. **One file** is the **report in pdf or docx file** (with a maximum total of FIVE pages). The second file is **PartA.ipynb** file of your work in Part A. The other files are **PartB_30_features.ipynb** or **PartB_9_features.ipynb**.

Notes: You should use **jupyter notebook** or **Google Colab** for developing your code; use a python framework (e.g. pytorch, sklearn, keras).

- An **abstract** is a concise overview of a document, including its background/purpose, methods, results, and conclusions.
- An abstract is designed **to help readers quickly decide if they need to read the full text.**
- A **summary** is a condensed version of the document's central ideas, focusing primarily on the main points and conclusions.
- A summary is used as **a standalone overview for readers who won't be reading the original document.**
- While an abstract is always a type of summary, a summary is not necessarily an abstract.

What is a good assignment submission ?

- Good abstract
- Good analysis of results
- Good presentation of results by table or graph (no need for python code or redundant convergence curves).
- Good discussion of your results. It means observation, insight or interpretation of your result and data.

Marking criteria (8%)

- Approach to the problem: different ANN structure (2)
- Technical results obtained and analysis (3)
- Presentation of results and discussion (3)